

Data Center Management - 2

Feb 11 - Day 1

Datacenter Networking 1 :

Computer Networks in 75 min

What is the internet?

- Vast, interconnected system of computer networks that span the globe
- Communication infrastructure
 - enables distributed applications
 - WEB, VoIP, email, games, e-commerce, etc.
- Internet "Network of networks"
 - loosely hierarchical
 - runs by protocol (common language that everyone understands)
 - two people able to understand each other

History

- 1960-1970s ARPANET, funded by US department of defense sites.
- 1980s-1990s: more networks joined, world wide web invented
 - int is underlying network of computers/connections
 - WWW

How int Works:

- Data packets: information traveling across internet is broken into small chunks called "packets" Each contains
 - A portion of the actual information being sent (part of email or piece of video)
 - Address information (like a return address on an envelope) telling routers where to send the packet
- server + clients:
 - A server is a computer that provides services or resources to other computers - like hosting websites, files, or email accounts.
 - a client is a computer/device that accesses those services
- Each packet includes

- source destination IP, protocol
- time to live (TTL)
- data

Internet protocol stack

- application layer: FTP, SMTP, HTTP
 - supporting network applications
- Transport layer: TCP, UDP
 - process to process data transfer
- Network layer: IP
 - routing of datagrams from source to destination
- Link Layer: Ethernet, wireless
 - data transfers between neighboring network devices
- Physical layer
 - bits on wires
- Encapsulation: Each layer wraps data with its own header

Physical Layer

- Wires, fibers, cables, radio waves: two devices to communicate with each other

How to enable multiple devices to communicate using a single wire:

Link Layer:

- Link Layer (MAC layer)
 - Logical network component used to interconnect nodes in the network
 - allow devices on a single wire to communicate with each other (LAN)
 - Bridging the network nodes (IP addresses) with physical devices (routers/switches)
- Medium access control (MAC): coordinating access of the shared medium
 - Unique hardware identifiers for devices, although modern software can report random MAC addresses
 - format: 48 bit address
- Protocols:
 - IEEE 802.3

Network Layers

- IP addresses to identify devices in other local area networks
- key functions of network layer

- forwarding: move packets from router's input to appropriate output
- Routing: Determine the route taken by packet from source to destination
- Data plane
 - local, per-router function
 - determines how arriving area is forwarded
- Control Plane
 - Network-wide logic
 - Determines how data is routed from source to destination
 - two approaches: Routing algorithms, software-defined networking (SDN)

Transport layer

- user datagram protocol (UDP)
 - point to point one sender, one reciever
 - barebones protocol: Lost or out of order
 - connectionless: no handshaking
 - fast
 - can be made reliable (user efforts)
- Transmission control protocol (TCP)
 - Point-to-point: one sender one reciever
 - reliable in order byte stream
 - popelined: congestion and flow control
 - full duplex data: bi-directional data flow
 - connection oriented: handshaking

Application Layer

- In cloud, users may have control over the application layer
- IaaS and PaaS
 - users specify desired protocols depending on application
- SaaS
 - cloud providers specify desired protocols

Scale of datacenter

- datacenter is large facility that hosts ~100k servers with
 - 100s of petabytes of storage
 - 100s of terabits of bandwidth
 - 10-100mw of power
 - cost \$1-4 billion to build

- Datacenter traffic growth

Switch vs router

- Switch
 - layer 2 devices (link layer) to forward data within a single network
 - operate based on MAC addresses, self learning what devices are connected to which part
- Router
 - Layer 3 devices to route packets between different networks

Top of rack architecture

- Rack of servers
 - High end commodity servers
 - top of rack switches
- Modular design
 - pre configured racks
 - power, network, and storage cabling

Why ToR?

- Less clutter
 - Connections are within each server cabinet (No messy cables)
 - save cost on cabling
- Flexibility in placement
 - can be placed anywhere in the cabinet(shorter cabling)
- Easy to rearrange
 - each cabinet is self-contained
 - quick rearrangement within datacenter
- efficient troubleshooting

Why. not ToR?

- Increased nubmer of switches
 - required one or more switches per rack, leading to higher total device count

End of row architecture

- Rack of servers
 - high end comodity servers
 - no switches in rack

- Modularize by functionality
 - servers and storage in rack
 - switches in separate rack

Keep building up

- Access layer switch
 - Commodity switches
 - commercial level (more expensive than consumer switch)
- Aggregation switches1.
 - datacenter level switch
- Core switches/Routers
 - datacenter-level switch

ToR vs EoR Comparison

Keep building up

- Access layer switches: 1GigE switches
- Commercial level (more expensive than consumer switch)
- Aggregation switches
- Datacenter-level switch: 10 GigE
- Core switches/routers
- Datacenter-level switch: 10GigE

3 tier architecture

Datacenter Network Layers

- Layer 3, IP addresses are distributed based on the host's subnet and cannot be the same once the Vm is migrated to a different subnet
 - Scalability through hierarchical addressing
 - efficiency through shortest path routing
 - multi path routing through equal-cost multipath
- In Layer 2, forwarding is performed based on MAC addresses and IP addresses can remain unchanged
 - cheaper switch equipments
 - fixed addresses and auto-configuration
 - seamless mobility, migration, and failover

Datacenter Traffic Patterns

- Modern datacenter has been a traffic pattern shift from north to south to east to west

- north south = SaaS, application oriented
- East West PaaS/IaaS, service oriented
- Two main reasons
 - storage co-locations
 - higher-degree of virtualization

Data Center Networking 2

- Single Administrative domain
 - Change all endpoints, switches, and routers if you want
 - limited interfaces with the outside world
- Unique network property
 - tiny round trip time (RTT) typically in microseconds
 - massive multipath topologies
 - Thin layer of switches and routers (little buffer)
- Latency and tail latency critical
 - Tail latency refers to the high end (or worst case) response times in a system's latency distribution
 - Tail latency: High %ile (eg 99.9) of a latency distribution
 - Network is a backplane for large-scale parallel computations

Delay in networking

- In networking, delays refer to the time taken for data to traverse the network from source to destination
- Processing Delay:
 - The time taken by network devices (like routers and switches) to process a packet's header, perform error checking, determine the packet's route, and execute any other necessary processing.
 - small in datacenter networking: DCN runs on industry grade switches and routers (expensive but powerful)
- Transmission Delay:
 - The time required to push all of a packet's bits onto the transmission medium (wire)
 - heavily depend on link bandwidth
 - small in datacenter networking: DCN uses high bandwidth cables and fibers

Delay in networking 2.0

- Propagation Delay:
 - the time it takes for a signal to travel from the sender to the receiver over the physical medium (copper cable, fiber optic, or wireless link)
- Queuing Delay:
 - time a packet spends waiting in a queue at a router or switch before it can be transmitted onto the next link
 - highly variable and dependent on network traffic load and congestion. During periods of heavy traffic, queuing delays can be a major contributor to overall latency.
 - huge in datacenter networking: datacenter has to handle heavy traffic cannot afford packets

Goal of DCN - agility

- Resource pooling
 - turn all the servers in a datacenter into a single large pool of resources
 - dynamically expand and contract services as needed
- Place workloads where server resources are available
 - the ability to utilize any server in DC without any networking constraints
- Easier to maintain availability
 - if one server/rack goes down, machines from other places should still be available
- DCN serves as one big high-speed fabric that connects all available resources

To Achieve this..

- Massive bisection bandwidth
 - bandwidth between any two halves of the network across a cut
 - topologies addressing, routing (multiple paths -> load balancing)
- Ultra-low latency (<10 microseconds)
 - transport protocol? switch scheduling? Buffer management?
 - schedule packets or control transmission rates
 - centralized or distributed control
- Effective resource management
 - multi-tenant performance isolation network isolation
 - App aware packet or flow scheduling

Simple (nm) port switching

- Different design of switching fabric
- assume n ingress ports and m egress ports
- desired architecture: any port can connect to any other port directly if all other ports are free

- Nonblocking: if input port x and output port y are both free, they should be able to connect
 - regardless of whether other ports are being used
 - if this is not satisfied, then it is blocking (Bad!)

3-stage clos network

- if $m \geq 2n-1$ then clos network is strict-sense nonblocking
 - $n-1$ existing demand at input and output
- any new demand between any pair of free (input, output) ports can be satisfied without re-routing any of the existing demands
- if $m \geq n$, then the clos network is rearrangeably nonblocking
- any new demand between any pair of free (in/output) ports can be satisfied by suitably re-routing existing demands
- $m \geq n$ is sufficient

Apply this to DCN

- switches are usually n by n with full duplex links
- fold the 3 stage clos into 2 stages with multiple switching plan
- share physical resources between ingress and egress stages
- share ports and links across the two sides of the middle stage
- this results in two stage high throughput datacenter topology
 - we can use the same switches (port density and link rates)
- But DCN is running w two tiers of 2 stage clos since server pods and spine planes are perpendicular to each other

What about Routing?

- We said that the Clos topology above is rearrangeably non-blocking
- so how to rearrange existing demands when a new packet arrives so that it can get across as quickly as possible
- how to do it without interference (rerouting) other pkts?
- In fact, we don't need to rearrange anything
 - switches and routers are fast enough to perform switching
- load balancing is the way to go

Valiant Load balancing (VLB)

- VLB is a randomized routing strategy originally proposed by Leslie Valiant
 - designed to evenly distribute network traffic across available paths, thereby mitigating congestion and avoiding hotspots in the network

- this tech is edsp valueable in high performance computing ,large datacenters and parralell processing systems
- Phase 1 (intermediate hop): when a packet is sent from a source to a destination, it is first routed to a randomly selected intermediate node within the network
- phase 2 (final delivery) once the packet reaches this intermediate node, it is then forwarded to the final destination
- guarentee: with high probability, the message reaches its destination very quickly
 - practically this means ver little queuing in the network

pt 2

- Randomization benefits
 - by randomly selecting the intermediate node, vlb disperses traffic more uniformly across network.
 - randomness helps revent scenarios where certain network paths or nodes become overloaded, which is a common problem in deterministic routing algorithms under worst-case traffic conditions
- VLB requirements: hose model
 - guarentees VLB + clos only hold under the hose model: demands for any one ToR port (send or recieve) must not exceed badwidth
 - very hard to enforce esp on the reciever slide without senderslide rate limits

Day 2: Feb 23

Valiant load balancing (VLB)

- randomized routing strategy originally proposed by leslie valiant
 - designed to evenly distribute network traffic across all available paths, thereby mitigating congestion and avoiding hotspots in the network
 - This technique is especially valubable in high-performance computing, large-scale data centers, and parallel processing systems
- Phase 1 (intermediate hop): when a packet (or unit of data) is sent from a source to a destination, it is first routed to a randomly selected intermediate node within the network
- Phase 2 (final delivery): once the packet reaches this intermediate node, it is then forwarded to the final destination
- Guarantee: with high probabiliyt, the message reaches its destination very quickly
 - practically, this means there is very less queueing in the network.

The network planes:

- Data plane: processing and delivery of packets with local forwarding state
 - Forwarding state + packet header -> forwarding decision
 - Filtering, buffering, scheduling
- control plane: computing the forwarding state in routers
 - determines how and where packets are forwarded
 - routing, traffic engineering, failure detection/recovery

Software defined networking:

- internet network layer: historically has been implemented via distributed per router approach.
 - monolithic router contains switching hardware, runs proprietary implementation of internet standard protocols (IP, RIP, IS-IS) in proprietary router OS (cisco IOS)
 - different "middleboxes" for different network layer functions: firewalls, load balancers, NAT boxes...
 - 2005: renewed interest in rethinking network plane

Recall: Per-router control plane

individual routing algorithm components in each and every router interact with each other in control plane to compute forwarding tables

Traffic engineering

- what if network admin wants to split a A-F traffic along ABCF and ADEF (load balancing purposes)
- Need to redefine link weight so traffic routing algorithm computes reroutes (or adopt new routing algorithm)

Traffic engineering is difficult (with per router control plane)

Logically centralized control plane

- a distinct (typically remote) controller interacts with local control agents (CA's) in routers to compute forwarding tables.

This is SDN

- Initially, software-defined networking =
 - separation of control and data plane
 - centralization of control
 - openFlow to talk to the data plane
- Why a logically centralized control plane?

- easier for network management: avoid router misconfigurations, greater flexibility of traffic flows
- Table-based forwarding (open flow API coming up shortly) allows programming routersL 1 centralized programming easier: compute tables centrally and distribute, 2 distributed "programming" more difficult and compute tables as result of distributed algorithm (protocol) implemented in each and every router
- open implementation of control plane
- enables and eases innovation
- Now the definition has changed significantly

Software defined networking

- Three features that define SDn
 - Abstract the hardware: no dependence on physical infrastructure. software APIs
 - Programmable: shift away from static manual operation to fully configurable and dynamic
 - centralized control of policies: policy delegation and management
- the wave of SDx
 - Software defined networking (SDN)
 - Software defined data center (SDDC)
 - Software defined storage (SDN)
 - Software defined compute (SDC)
 - Software defined infrastructure

Generalized Forwarding and SDN

each router contains a flow table that is computed and distributed by a logically centralized routing controller

Open flowL flow table entries:

Open flow abstraction:

flow: defined by header fields

generalized forwarding: simple packet-handling rules

pattern: match values in packet header fields

actions: for matched packet: drop forward, modify matched packet or send matched packet to controller

priority: disambiguate overlapping patterns

counters: # bytes and # packets

Match+action: Unifies different kinds of devices

- router: match -> longest destination IP prefix; action -> forward out a link

- switch: match -> destination MAC address; action -> forward or flood
- firewall: match -> IP addresses and TCP/UDP port numbers; action -> permit or deny

Building up...

control layer

- maintain network state information
 - interacts with network control applications "above" via northbound API
 - interacts with network switches "below" via southbound API
 - Implemented as distributed system for performance, scalability, fault tolerance, robustness
- Application layer
- brains of control: implement control functions using lower-level services, API provided by SDN controller
 - unbundled: can be provided by 3rd party: distinct from routing vendor, or SDN controller

SDN Controller = network OS

- network OS (NOS): system that creates a consistent, up to date network view, with distributed cores
 - runs on servers (controllers) in the network
 - NOX, ONIX, floodlight, Trema, OpenDaylight, HyperFlow, Kandoo, Behave
- uses forwarding abstraction to:
 - Get state information from forwarding elements
 - give control directives to forwarding elements

OpenDaylight (ODL)

- Network apps may be contained within, or be external to SDN controller
- Service abstraction layer: interconnects internal, external applications and services

Open Network Operating systems (ONOS)

- Control apps separate from controller
- considerable emphasis on distributed core
 - service reliability
 - replication
 - performance
 - scaling

The Road to SDN

- Active networking: 1990s

- first attempt make networks programmable
- demultiplexing packets to software programs, network virtualization
- control/dataplane separation: 2003-2007
 - ForCes, RCP, 4D
 - Open interfaces between data and control plane, logically centralized control.
- OpenFlow API and Network OSes: 2008
 - OpenFlow switch interface standard
 - NOX Network OS

SDN Drives

- rise of merchant switching silicon
 - democratized switching
 - vendors eager to unseat incumbents
- cloud/datacenters
 - operators face real network management problems
 - extremely cost conscious; desire a lot of control
- The right balance between vision and pragmatism
 - openflow compatible with existing hardware
- A "Killer app" : network virtualization
 - divide global network into small local network
 - network applications run on virtual topology

Network Virtualization:

Consider a multi-tenant datacenter

want to allow each tenant to specify virtual topology

this defines their individual policies and requirements

- Datacenter's network hypervisor compiles these virtual topologies into set of switch configurations
 - takes 100s of individual tenant virtual topologies
 - Computes configurations to implement all simultaneously
- This is what people are paying money for..
 - Enabled by SDN's ability to virtualize the network.

SDN in datacenter

- Virtualization: use network resource without worrying about where it is physically located, how much it is, how it is organized, etc.

- orchestration: should be able to control and manage thousands of devices with one command
- programmable: should be able to change behavior on the fly.
- dynamic scaling. should be able to change size
- automation: to lower OpEx minimize manual involvement, e.g. troubleshooting, reduce downtime, policy enforcement, etc.
- visibility: Monitor resources, connectivity

SDN in DataCenter

- performance: optimize network device utilization, e.g, traffic engineering/bandwidth management, capacity optimization, load balancing, high utilization
- multi-tenancy: tenants need complete control over their addresses , topology, and routing, security,
- service integration: load balancers, firewalls, intrusion detection systems (IDS), provisioned on demand and placed appropriately on the traffic path
- openness: full choice of "HOW" mechanisms -> modular plug-ins.

Feb 25

Datacenter packet transport

Two types of workload in DC

- Mice: short messages (query, coordination) -> low latency
- elephant: large flow (data update, backup) -> high throughput
- coexistent of the two workloads creates performance impairments requirements
- high throughput
- low latency
- burst tolerant

DC is deadline driven

- a deadline refers to the strict timeframe by which a project must be completed, often tied to contractual obligations with clients while "profit" represents the financial gain made from operating the data center once it is fully operational

Deadline-based flow scheduling

- networking should never be the bottleneck of completing a task in datacenter
 - ensuring that data flows meet their deadlines can significantly improve the overall responsiveness and performance of applications hosted in DC

Head of line blocking:

- resource contention occurs in the core of the network
- Congestion control (CC) will react, but may be too little and too late
 - congestion control can only reduce possibility of future packets being dropped; not prevent packet drop now
 - congestion cannot prevent mouse flow from inflicting large delays and recurring drops
- reduce timeout will only make this worse

Shared memory buffering

- packets sent to the switch/router that exceed speed of service need to go somewhere to wait for their turn
 - input queue vs output queue
- Buffer management: how to put packets into the queue
 - single shared queue? priority?
 - static buffer allocation vs dynamic buffer allocation
- scheduling: how to serve packets in the queue
 - first in first out. FIFO, priority first? round robin?

Recall: TCP congestion control

- keep some in flight (un acked) packets: congestion window
- adjust window based on several algorithms -tcp new reno
 - startup: slow start
 - steady state: AIMD additive increase/multiplicative decrease
 - loss: fast retransmission, fast recovery

DCTCP

- datacenter TCP is a congestion control algorithm specifically designed to address the unique needs of data center networks
- goal: extract multi bit feedback from single bit stream of ECN marks
 - reduce window size based on fraction of marked packets.

DCTCP Algorithm

- switch side: mark packets when queue length $> K$

- Sender side
 - reacting to and controlling queue size distribution
 - maintain running average of fraction of packets marked (a)
 - each RTT: $F = \# \text{ of marked acks} / \text{total } \# \text{ of acks} = a = (1-g)a + f * F$
 - adaptive window decreases: decrease factor between 1 and 2
 - If no congestion is observed a remains low and reduction is minimal
 - if congestion is severe a approaches 1 the window is reduced by nearly 50%

Delayed ACKs

- not every packet is ACKnowledged by receiver
- too many ACKs: increase packet processing load
 - typical policy: ACK every m packets, or after sender has paused transmitting for a delayed ACK timeout
- How to allow the sender to see the full stream of ECN marks?

Why it works

Low latency

- small buffer occupancies -> low queuing delay
high throughput
- ECN averaging -> smooth rate
Burst tolerance
- large buffer headroom -> bursts fit
- aggressive marking -> sources react before packets are dropped

BOLT: sub RTT congestion control

- bandwidth delay product is increasing
- remote procedure call
 - high performance request response protocol allowing a client program to execute functions on a remote server as if they were local
 - acts as a primary communication mechanism for distributed systems

Ideal congestion control:

detect precise amount and location of congestion

- fine grained feedback
 - act instantaneously
 - minimizing the feedback delay
 - congestion notification delay
 - under utilization feedback delay

March 2

Bolt: Sub-RTT Congestion control

- bandwidth-delay-product (BDP)
**come back

Ideal congestion control

- detect precise amount and location of congestion
 - fine-grained feedback
- act instantaneously
- minimizing the feedback delay
 - congestion notification delay
 - under-utilization feedback delay

Minimizing congestion notification

Decrease congestion notification delay to its absolute minimum

True/false

DCTCP uses an adaptive congestion window decrease, adjusting its running window based on the fraction of packets marked

Public v private

Public = computing environment where services like servers, storage, applications are provided by third party vendors over internet

Ownership + management: managed by third party providers

resource sharing infrastructure is shared among multiple users - bad

scalability + cost: highly scalable with. a pay as you go pricing model, typically more cost-effective

maintenance: providers handle system updates, security and maintenance.

accessibility: availability of anyone over the internet making it widely accessible

Private:

cloud computing environment exclusively dedicated to single organization

ownership + management: dedicated to single organization, can be managed internally or by third party.

isolation + security: offers enhanced security with isolated resources tailored to specific needs

Customization: provides greater control over infrastructure - bad

allowing for customized configurations good

cost = generally higher costs due to dedicated hardware and maintenance responsibilities - bad
compliance: ideal for organizations with strict regulator and compliance requirements